RICHARD JOHNSON
ANGLICAN COLLEGE
93 Hyatts Road Oakhurst NSW 2761 | 02 9677 2455

Assessment Notification

Year: 12 Subject: Software Engineering Task 3

Task title	**Software Engineering Project - DRAFT**		
Date issued	Monday, 23 March 2026	Weighting	30%
Due date	Monday, 8 June 2026	Total marks	50

Outcomes:

- **SE-12-01** justifies methods used to plan, develop and engineer software solutions
- **SE-12-07** designs, develops and implements safe and secure programming solutions
- **SE-12-06** justifies the selection and use of tools and resources to design, develop, manage and evaluate software
- **SE-12-09** applies methods to manage and document the development of a software project

Context:

In this task, you will act as a Lead Engineer tasked with modernising a legacy codebase. You will take the web application you built in Term 4 and upgrade it into an **Enterprise-Ready, Data-Driven Application**.

Your Term 4 project was a prototype. It proved your concept worked. Now, you must fundamentally evolve that prototype into a secure, intelligent software platform. Your goal is twofold:

1. **Secure Software Architecture:** Conduct a rigorous security audit of your Term 4 code against the OWASP Top 10. You will use SAST (Static Application Security Testing) and DAST (Dynamic Application Security Testing) methodologies to identify vulnerabilities and implement reliable improvements to your app's structure that protect it from security problems.
2. **Software Automation:** Integrate a machine learning regression model (e.g., Linear, Logistic, or Polynomial Regression using libraries like *scikit-learn*) to automate a data-driven prediction or classification feature that fundamentally enhances the core utility of your application.

You are not just adding features; you are upgrading the entire system architecture. You will rely heavily on visual systems modelling (e.g. Data Flow Diagrams) and automated version control (GitHub) to document your journey, entirely replacing written reports and logbooks.

Your final, upgraded solution must be deployed live on **PythonAnywhere**.

Description of the task:

PART A: Planning, Security & Modelling (25 Marks)
Your theoretical blueprints, security audits, and project management evidence. These will form your printed Project Documentation.

1. System Blueprint (5 Marks)

- **1.1 System Upgrade & Justification (3 marks):** Provide 2-3 sentences defining the specific task your new automation feature will perform. You must explicitly identify the Regression Model chosen (e.g., Linear, Logistic, Polynomial) and **justify** why it is the most appropriate mathematical approach for your specific dataset and desired outcome.
- **1.2 Context Diagram (Level 0 DFD) (2 marks):** Provide a single diagram showing the new system boundaries, highlighting the flow of data through the new automation component and security checkpoints.

2. Threat Model & Security Plan (5 Marks)

- **2.1 OWASP Vulnerability Assessment Table (3 marks):** Identify two specific weaknesses in your Term 4 code (e.g., "Search bar vulnerable to SQL injection") and explicitly state the architectural technique you will use to patch them (e.g., "Implement parameterised queries via SQLAlchemy").
- **2.2 SAST & DAST Testing Plan (2 marks):** Provide a bulleted list defining how you will use Static Application Security Testing (e.g., Bandit, Flake8) and Dynamic Application Security Testing (e.g., manual penetration testing) to verify your app's security.

3. Iterative Project Management (5 Marks)

- **3.1 GitHub Commit History (3 marks):** A minimum of 15 incremental commits with descriptive messages explaining your refactoring and testing process (e.g., *"Ran SAST tool and patched SQLi vulnerability"*, *"Trained Logistic Regression model on user data"*). This serves as your official project logbook.
- **3.2 Gantt Chart (2 marks):** A visual timeline mapping your 10-week refactoring sprint, explicitly showing the phases for security patching, machine learning integration, and testing.

4. Data Flow Modelling (5 Marks)

- **4.1 Data Flow Diagram (Level 1 or 2 DFD) (5 marks):** Provide a highly accurate, industry-standard diagram mapping how data moves from the user interface, through your new security validations, into the regression model for processing, and how the output is returned. It must use correct DFD symbols.

5. Logic & Architecture (5 Marks)

- **5.1 Flowchart or Algorithm Design (3 marks):** Design a clear flowchart or structured algorithm representing the core Python logic for your machine learning/regression integration, demonstrating how the data is ingested, processed, and evaluated.
- **5.2 Structure Chart OR Database Schema (2 marks):** Produce a diagram showing the updated module hierarchy of your Flask application, OR provide an updated Database Schema detailing how your tables have been secured and restructured from Term 4.

PART B: Secure Development, Automation & Deployment (25 Marks)
The fully refactored, integrated, and deployed Python/Flask web application.

6. Automation Functionality (5 Marks)

- **6.1 Regression Model Execution (3 marks):** The Python automation feature successfully uses a regression model (Linear, Logistic, or Polynomial) using appropriate libraries (e.g., scikit-learn). It processes the provided dataset without crashing or throwing server errors.
- **6.2 Output Accuracy & Relevance (2 marks):** The model generates a logical prediction, classification, or automated output that genuinely enhances the application's utility and solves the objective defined in Part A.

7. Web Integration (5 Marks)

- **7.1 Flask Routing & Data Handling (3 marks):** Data is successfully and securely passed from the web form to the Python backend and back. Route handlers are logically structured and manage GET/POST requests appropriately.
- **7.2 Seamless UI Integration (2 marks):** The automation feature feels like a natural extension of the Term 4 app. The model's predictions and automated results are dynamically and clearly rendered to the user interface, providing a smooth user experience.

8. Secure Architecture (5 Marks)

- **8.1 Security Patch Implementation (3 marks):** The vulnerabilities identified in Part A have been successfully patched. The application features robust OWASP mitigations, specifically preventing SQL Injection (SQLi) and ensuring safe data handling.
- **8.2 Testing Evidence (2 marks):** Clear evidence of SAST/DAST testing having been conducted must be visible in the code's evolution (e.g., demonstrated via specific GitHub commit messages or detailed code comments explaining the security patches).

9. Code Quality (5 Marks)

- **9.1 Code Readability & Comments (3 marks):** The refactored code includes excellent, meaningful comments that clearly explain the complex new security implementations and the mathematical logic of the machine learning integration.
- **9.2 Standards & Cleanup (2 marks):** The Python code strictly adheres to PEP 8 formatting standards. Redundant, messy, or insecure legacy code from the Term 4 build has been successfully cleaned up and refactored.

10. Deployment & Presentation (5 Marks)

- **10.1 Live Hosting (2 marks):** The upgraded Flask application must be successfully deployed, fully functional, and accessible via a live URL on PythonAnywhere.
- **10.2 Digital and Printed Presentation (2 marks):** Your Project Documentation (containing all items from Part A) must have a spelling and grammar check and be presented with a table of contents, header, footer, headings, subheadings, screenshots, tables, and appropriate visuals. This must be uploaded as a single Word document and submitted as a printed and bound document.
- **10.3 GitHub README.md (1 mark):** Provide a README.md file in your GitHub repository containing clear instructions on how to access and use the app, along with a brief "Security Patch & Automation Notes" summary for the user.

Marking criteria: Refer to the marking scaffold below. All written work must be the student's original creation. Failure to cite sources, including AI-generated content, will be considered academic malpractice and may result in penalties such as reduced marks, a zero grade, or further disciplinary action per the school's academic integrity policy.

Feedback provided: Throughout the unit, you will receive either written or verbal feedback. During discussions, the teacher will highlight your strengths and areas that require further development.

Submission Guidelines

1. **Document Submission:** Submit via Canvas a single, professionally formatted Word document containing all required blueprints, threat models, and system diagrams (Parts A and B). A printed and bound copy of this identical document must also be submitted in person by **3:10PM on the due date.**

2. **Links & Hosting:** The title page of your Word document must clearly display:
   - The live URL to your deployed web application on **PythonAnywhere**.
   - The link to your **private GitHub repository**.

3. **Code Submission:** All refactored Python code, system files, and your README.md must be pushed to your private GitHub repository. Your GitHub commit history will be assessed as your official project logbook. As a backup, you must also submit a .zip file containing all code and project files to Canvas.

4. **Academic Integrity & AI:** Any use of Generative AI for code generation, debugging, or threat modelling must be clearly and explicitly acknowledged. Because there is no traditional written logbook, AI assistance must be cited directly in your **GitHub commit messages**, your **README.md file**, and the **Bibliography** of your Word document.

Use of AI
Please refer to the College 'Use of AI Technology Policy'

You may use AI for planning, idea development, and research. Your final submission should show how you have developed and refined these ideas.

APA Referencing Requirements

All research and written components of this task must include APA 7th edition referencing.
Students are required to:

Acknowledge all sources used for information, images, data, quotes, and AI-assisted material.

Include in-text citations whenever information is paraphrased or quoted.

Provide a reference list on a separate page at the end of the task.

Ensure references follow correct APA 7th edition format, including author, year, title, and source.

AI-generated content must be referenced, indicating the prompt and the AI tool used (e.g., ChatGPT, 2025).

Failure to correctly acknowledge sources may result in a loss of marks for academic integrity.

Faculty Leader approval:

Students are reminded of the rules and requirements relating to completion, submission and absences for assessment tasks.	**Faculty Leader approval**

Year 12 Software Engineering: Software Engineering Project – Task 3 Marking Scaffold

CRITERIA	WORK ASPECT	RATINGS					
SE-12-01 justifies methods used to plan, develop and engineer software solutions **SE-12-06** justifies the selection and use of tools and resources to design, develop, manage and evaluate software **SE-12-09** applies methods to manage and document the development of a software project	**PART A: Planning, Security & Modelling (25 Marks)**						
	1.1 System Upgrade & Justification (3 marks)			**3**	**2**	**1**	**0**
				(Extensive / Thorough): Identifies the model and provides a highly logical, data-driven justification for why it fits the dataset and objective.	**(Sound):** Identifies the model and objective, but the justification is basic, slightly vague, or lacks a strong mathematical connection to the data.	**(Basic / Elementary):** Names a regression model but provides no meaningful justification for its use.	**(Minimal / Non-Submission):** Missing, not attempted, or fundamentally flawed.
	1.2 Context Diagram (2 marks)				**2**	**1**	**0**
					(Extensive / Thorough): Perfect Level 0 DFD accurately showing system boundaries, data flow, and security checkpoints with correct syntax.	**(Sound / Basic / Elementary):** Diagram is present but contains minor symbol errors or misses a checkpoint.	**(Minimal / Non-Submission):** Missing, not a context diagram, or fundamentally illogical.
	2.1 OWASP Vulnerability Table (3 marks)			3	**2**	**1**	**0**
				(Extensive / Thorough): Accurately identifies 2 weaknesses and explicitly states valid, robust architectural patches (e.g., parameterised queries).	**(Sound):** Identifies 2 weaknesses but patches are vague (e.g., "fix passwords"), OR only identifies 1 weakness with a solid patch	**(Basic / Elementary):** Identifies a weakness but provides no architectural solution, or patches are entirely incorrect.	**(Minimal / Non-Submission):** Missing or not attempted.
	2.2 SAST & DAST Testing Plan (2 marks)				**2**	**1**	**0**
					(Extensive / Thorough): Clear bulleted list defining how *both* Static and Dynamic testing will be explicitly used.	**(Sound / Basic / Elementary):** Mentions testing but is vague, or only focuses on one type of testing (e.g., only manual testing).	**(Minimal / Non-Submission):** Missing or not attempted.
				3	**2**	**1**	**0**

	3.1 GitHub Commit History (3 marks)			**(Extensive / Thorough):** 15+ incremental commits. Messages are highly descriptive and clearly explain the technical refactoring/testing process.	**(Sound):** 10–14 commits, OR 15+ commits but messages are consistently vague (e.g., "updated code", "fixed bug").	**(Basic / Elementary):** Fewer than 10 commits, or appears to be a single "bulk upload" at the end of the project.	**(Minimal / Non-Submission):** No GitHub repository provided.
	3.2 Gantt Chart (2 marks)				**2**	**1**	**0**
					(Extensive / Thorough): Highly logical visual timeline accurately mapping the 10-week sprint, including ML and security phases.	**(Sound / Basic / Elementary):** Timeline is present but highly basic, unrealistic, or misses major development phases.	**(Minimal / Non-Submission):** Missing or not attempted.
	4.1 Data Flow Diagram (5 marks)	**5**	**4**	**3**	**2**	**1**	**0**
		(Extensive): Flawless Level 1 or 2 DFD. Expertly maps data through security validations and the regression model using perfect syntax.	**(Thorough):** Excellent logic and flow, but contains a single minor syntax or labelling error.	**(Sound):** Sound logic, but contains a few noticeable syntax errors or slightly unclear data pathways.	**(Basic):** Basic attempt. Demonstrates a poor understanding of how data moves through the new system logic.	**(Elementary):** Elementary attempt. Uses the completely wrong diagram type or is barely readable.	**(Minimal / Non-Submission):** Missing or not attempted.
	5.1 Flowchart or Algorithm Design (3 marks)			**3**	**2**	**1**	**0**
				(Extensive / Thorough): Accurately and logically represents the core	**(Sound):** Mostly accurate but misses minor logical steps,	**(Basic / Elementary):** Highly basic, confusing, or does not align with	**(Minimal / Non-Submission):**

				Python/ML logic, including data ingestion and processing.	edge cases, or decision diamonds.	the actual coded solution.	Missing or not attempted.
	5.2 Structure Chart OR Database Schema (2 marks)				**2**	**1**	**0**
					(Extensive / Thorough): Accurate, clearly labelled diagram effectively detailing the architectural or database upgrades.	**(Sound / Basic / Elementary):** Diagram is present but contains structural errors or lacks clear labelling.	**(Minimal / Non-Submission):** Missing or not attempted.
SE-12-01 justifies methods used to plan, develop and engineer software solutions **SE-12-09** applies methods to manage and document the development of a software project	**PART B: Secure Development, Automation & Deployment (25 Marks)**						
	6.1 Regression Model Execution (3 marks)			**3**	**2**	**1**	**0**
				(Extensive / Thorough): Python regression model runs flawlessly, processes input data without crashing, and handles errors gracefully.	**(Sound):** Model runs and processes data but contains minor bugs or occasionally throws handled errors	**(Basic / Elementary):** Code is present but frequently crashes or completely fails to process the dataset.	**(Minimal / Non-Submission):** Missing or not attempted.
	6.2 Output Accuracy & Relevance (2 marks)				2	1	0
					(Extensive / Thorough): Generates a highly logical, mathematically appropriate prediction/output that solves the stated objective.	**(Sound / Basic / Elementary):** Generates an output, but it is inaccurate, uses poor data, or lacks relevance to the app's utility.	**(Minimal / Non-Submission):** Missing or fails to generate output.
	7.1 Flask Routing & Data Handling (3 marks)			**3**	**2**	**1**	**0**
				(Extensive / Thorough): Data is successfully and securely passed between the frontend and Python backend.	**(Sound):** Routing works, but data handling is clunky, inefficient, or slightly disjointed.	**(Basic / Elementary):** Fundamentally broken routing; backend and frontend fail to communicate.	**(Minimal / Non-Submission):** Missing or not attempted.

				GET/POST routes are flawless.			
	7.2 Seamless UI Integration (2 marks)				**2**	**1**	**0**
					(Extensive / Thorough): Automation feels native. Results are clearly, dynamically, and beautifully rendered to the user interface.	**(Sound / Basic / Elementary):** Results are displayed, but the rendering is basic, awkward, or breaks the app's visual formatting.	**(Minimal / Non-Submission):** Missing or results only print to the backend console.
	8.1 Security Patch Implementation (3 marks)			**3**	**2**	**1**	**0**
				(Extensive / Thorough): Part A vulnerabilities are fully patched with robust, industry-standard OWASP mitigations (e.g., parameterised queries).	**(Sound):** Security measures are implemented but may be incomplete or can be theoretically bypassed.	**(Basic / Elementary):** Minimal to no real security improvements, or patches are entirely ineffective.	**(Minimal / Non-Submission):** Missing or not attempted.
	8.2 Testing Evidence (2 marks)				**2**	**1**	**0**
					2 Marks (Extensive / Thorough): Clear, undeniable evidence of SAST/DAST testing is visible in commit history or specific code comments.	**1 Mark (Sound / Basic / Elementary):** Vague mentions of testing with no concrete proof in the codebase or version control.	**(Minimal / Non-Submission):** Missing or not attempted.
	9.1 Code Readability & Comments (3 marks)			**3**	**2**	**1**	**0**
				(Extensive / Thorough): Excellent, meaningful comments that deeply explain the complex ML math and security implementations.	**(Sound):** Comments are present but are highly basic (e.g., "this is the model") and lack technical depth.	**(Basic / Elementary):** Sparse or irrelevant comments.	**(Minimal / Non-Submission):** Missing or not attempted.
					2	**1**	**0**

	9.2 Standards & Cleanup (2 marks)				**(Extensive / Thorough):** Strict adherence to PEP 8 standards. Redundant/messy Term 4 legacy code has been successfully cleaned out.	**1 Mark (Sound / Basic / Elementary):** Minor PEP 8 formatting issues, or some messy Term 4 code was left abandoned in the files.	**(Minimal / Non-Submission):** Highly disorganised code; no formatting standards applied.
	10.1 Live Hosting (2 marks)				**2**	**1**	**0**
					(Extensive / Thorough): App is successfully deployed, fully functional, and accessible via a live PythonAnywhere URL.	**(Sound / Basic / Elementary):** Deployed but experiences crashes, missing CSS/assets, or environment errors on the live server.	**(Minimal / Non-Submission):** Only runs locally; no live URL provided.
	10.2 Digital and Printed Presentation (2 marks)				**2**	**1**	**0**
					(Extensive / Thorough): Professional presentation (TOC, headers, visuals), flawless spelling/grammar, submitted in both formats (Word + Bound).	**(Sound / Basic / Elementary):** Minor spelling/formatting errors, missing one presentation element, or only submitted in one forma	**(Minimal / Non-Submission):** Significant formatting errors, unreadable, or not submitted.
	10.3 GitHub README.md (1 mark)					**1**	**0**
						(Extensive / Sound): Contains clear access instructions and a good security/automation patch summary.	**(Basic / Minimal / Non-Submission):** Missing, or contains no useful information for the end user.